The betaMixPy package

Haim Bar

June 2026

1 Introduction

The betaMixPy package is used to find edges in gene networks using co-expression data. The betaMix method is built on the insightful results of [1] who showed that two random vectors are approximately perpendicular with high probability, if the dimension of the space is sufficiently large. The pair-wise correlations between pairs of predictors are equal to the cosine of the angles between the pairs. From these angles we compute $z_j = \sin^2(\theta)$ and fit a mixture of two beta distributions. For pairs of random vectors (the null set) the distribution of z_j is Beta((N-1)/2, 1/2), where N is the sample size. The nonnull set is assumed to follow a Beta(a,b) distribution, and using the EM algorithm we estimate a,b, and the proportion, p_0, of the null set of pairs. The betaMix function determines a threshold which will control the error rate given by the user. Any z_j below that threshold corresponds to a significantly correlated pair of predictors (an edge in the graphical model.) If the N samples can be assumed to be independent, set the parameter `ind` to True. If it is set to False, the null set follows a Beta((ν-1)/2, 1/2) distribution and ν (the effective sample size) is estimated from the data.

The input to the program is a normalized expression matrix with **samples in rows and variables (genes/predictors) in columns** — matching numpy's `corrcoef(M, rowvar=False)` and the R `betaMix` package convention. `betaMix()` returns a Python dict whose keys (`angleMat`, `z_j`, `m0`, `p0`, `ahat`, `bhat`, `etahat`, `ppthr`, `nodes`, `edges`, `bmax`, `N`, `cnt`, `method`) mirror R's named list.

With a large number of predictors, P, the estimation may be slow, so it is recommended to set the parameter `subsamplesize` to something smaller than $P(P-1)/2$. The minimum allowed by the program is 20,000. Using anything smaller will cause betaMix to fit the model to all $P(P-1)/2$ pairs.

The betaMix package requires the following Python packages: statistics, matplotlib, pandas, networkx, numpy, scipy.

2 Simulated Data

The data folder in this repository contains four simulated datasets as csv files:

1. `cls_0200_1000_03_0050.csv` contains 1000 predictors, and $N = 200$ samples. The predictors come in 20 clusters, with 50 predictors in each cluster, such that the correlation of pairs within cluster is 0.3, and zero across clusters.

2. `rbk_0500_0500_03.csv` Similar to the previous item, except that the clusters have random sizes, and $P = 500$, and $N = 500$.

3. `hub_0500_0500_03_0025.csv` contains 500 predictors, and $N = 500$ samples. The predictors are arranged as 25 hubs, so that each hub consists of 20 predictors of which one is the central node and all other nodes have a 0.3 correlation with it. The correlation of pairs across hubs is zero.

4. `hub_0200_1000_09_0020.csv` Similar to the previous item, except that $P = 1000$, $N = 200$ and the correlation between a central node and all other nodes in a hub is 0.9.

We demonstrate the usage of the package with the last configuration (`i=3`, the random-block dataset) in the code below.

```
"""
Example: betaMixPy on simulated hub-structured data.

M convention: N rows = samples, P columns = variables (matches R betaMix).
```

```
The CSV files have samples as rows, so NO transpose is needed.
"""
import pandas as pd
import betaMixPy as bx
import matplotlib.pyplot as plt

prefix = "../data/"
files = [
    "cls_0200_1000_03_0050",
    "hub_0500_0500_03_0025",
    "hub_0200_1000_09_0020",
    "rbk_0500_0500_03",
]

i = 3
# Samples in rows, variables in columns (no transpose needed)
M = pd.read_csv(prefix + files[i] + ".csv", index_col=0)
print("Data shape (N, P):", M.shape)

res = bx.betaMix(M)
bx.shortSummary(res)
```

The CSV file has samples in rows, so no transpose is needed. We call `betaMix(M)` with default settings; the null posterior probability threshold defaults to `ppr=0.05`. Setting `ind=True` (the default) means the N samples are assumed independent and the null distribution is determined exactly by the sample size.

After fitting the model, we can obtain a summary of the parameter estimates and a plot of the fitted mixture distribution on top of the actual data histogram. This is done with the following code:

```
bx.plotFittedBetaMix(res)  # , file="../figures/fitted.pdf"
plt.close("all")
```

The model-fit summary in this case is summarized as follows:

```
Fitting the model...

No. Nodes: 1000
Possible no. edges: 499500
No. edges detected = 15742
Detection threshold: 0.92
Pr(null): 0.9588
Nonnull support: [0, 0.95 )
Nonnull parameters (a, b, eta): ( 65.28 ,  4.39 ,  99.5 )
```

The fitted distribution plot is shown on the log-scale (Figure 1), where the null distribution is linear (depicted as a light blue line.) The nonnull component (shown in purple) is obtained from the estimates `ahat`, `bhat`, and `bmax` (the upper bound of the support of non-null component). The red dotted line shows the cutoff, such that all the pairs which yield a z_j statistic less than this threshold are considered nonnull. In this case, all the nonnull pairs were generated with the same correlation so the beta distribution fits the nonnull component very well but, in general it doesn't have to be this way – the important thing is that for true nonnull pairs, the nonnull component will be much more likely than the null distribution.

In order to cluster the predictors we first have to generate the adjacency matrix and find graph components by running the following code

```
adjM = bx.getAdjMat(res)
df1  = bx.graphComponents(adjM, minCtr=2)
```

Then, we can show the pairwise correlations as a bitmap, as in Figure 2.

```
bx.plotBitmapCC(adjM, file="../figures/CorMatEst.pdf")
```

We can clearly see in Figure 2 that the algorithm detects the correct correlation structure (the yellow squares along the diagonal). The true correlation is modest ($\rho = 0.3$) and $P = 1000$ is quite large, and greater than $N = 200$, but still, most of the true correlations are detected. In

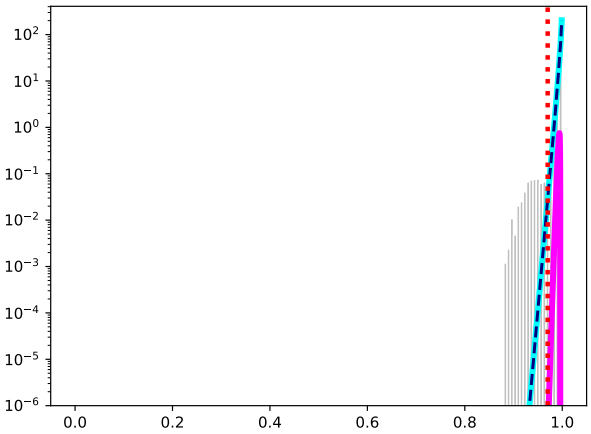


Figure 1: Simulation 1 – $N = 200$, $P = 1000$, 20 clusters with 50 predictors in each cluster, such that the true correlation of pairs within cluster is 0.3, and zero across clusters.

similar simulations, where either ρ is greater, or N is larger the true positive rate approaches 1. Furthermore, there are no yellow dots outside the diagonal of blocks, since our method yields an excellent false detection rate.

To visualize a certain cluster, we use the following code, which yields Figure 3.

```
bx.plotCluster(adjM, 1, df1, labels=True, nodecol="#DDFF00")
```

3 Case Study – Tomato Seeds

We use the tomato metabolomic data from www.metabolomicsworkbench.org website, Study ID ST000561, introduced in [3]. There are two groups – 'dry seeds' and 'six-hour imbibed seeds'. Each group has $N = 50$, and a total of $P = 68$ metabolites. We construct the metabolite network for

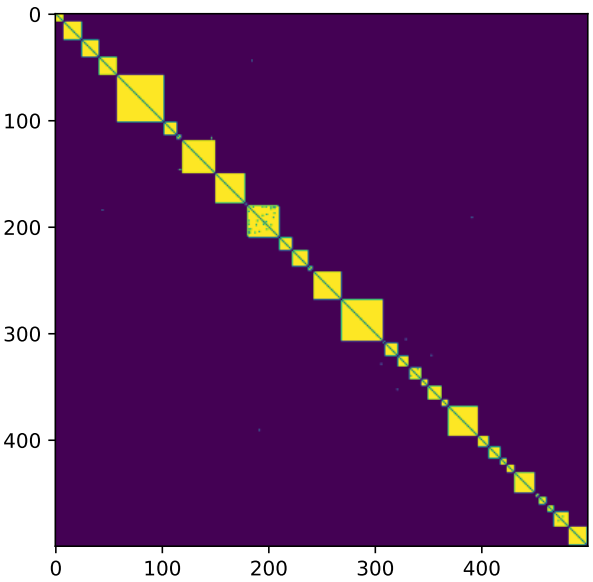


Figure 2: Simulation 1 – the inferred adjacency matrix after fitting the betaMix model.

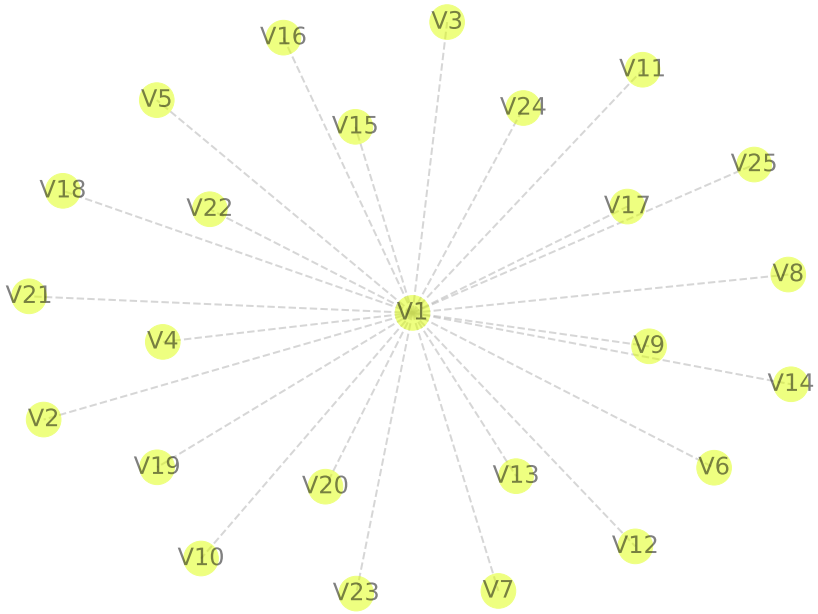


Figure 3: Simulation 1 – cluster 1.

each group separately. The code for the control (dry seeds) group is given below. The code for the second group is similar (and available in the GitHub repository.)

```
"""
Example: betaMixPy on metabolomics data (Dry Seeds and Six-Hour Imbibed).

M convention: N rows = samples, P columns = variables (matches R betaMix).
The CSV files have samples in rows, so NO transpose is needed.
"""
import pandas as pd
import betaMixPy as bx
import matplotlib.pyplot as plt

# --- Dry Seeds ---
M0 = pd.read_csv("../data/DrySeeds.csv", index_col=0)
print("DrySeeds shape (N, P):", M0.shape)

res0 = bx.betaMix(M0, subsamplesize=50000, ind=True, ppr=0.01)
bx.shortSummary(res0)
```

Figure 4 shows the bitmap representation of the two networks. The metabolites in the Dry Seed group are much less connected than in the Six-hour Imbibed group.

Figure 5 shows the cluster which contains the GABA metabolite in both groups. Both clusters are highly connected, but in the Dry Seed group (left) the cluster consists of five metabolites, whereas in the Six-hour Imbibed group four additional metabolites are found to be in the GABA cluster. All the edges in Figure 5(a) are also detected to be present in Figure 5(b).

3.1 Overlapping Communities via `greedyBasis`

Unlike `graphComponents` (which assigns each node to exactly one cluster) and `sphericalCaps` (which builds overlapping hub-and-spoke communities), `greedyBasis` finds a *maximal independent set* (MIS) of the network. A MIS is a subset of nodes such that (i) no two members share an edge (*independence*) and (ii) every node outside the MIS is adjacent to at least one member (*maximality*). After finding this basis, each node is assigned to every community whose centre it is adjacent to in the original graph, yielding overlapping membership.

The following code fits betaMix to the Dry Seeds group and then applies `greedyBasis` with both strategies to the resulting adjacency matrix.

```
M   = pd.read_csv("../data/DrySeeds.csv", index_col=0)
```

```python
res = bx.betaMix(M, ppr=0.01, ind=True, msg=False)
adjM = bx.getAdjMat(res)

print("=== strategy='min_degree' (classical MIS, non-redundant basis) ===")
bf_min = bx.greedyBasis(adjM, strategy="min_degree", labels=M.columns)
bx.summarizeBasis(bf_min)

print()
print("=== strategy='max_degree' (hub-first MIS) ===")
bf_max = bx.greedyBasis(adjM, strategy="max_degree", labels=M.columns)
bx.summarizeBasis(bf_max)
```

The output for the Dry Seeds network ($P = 68$ metabolites, 203 edges) is as follows.

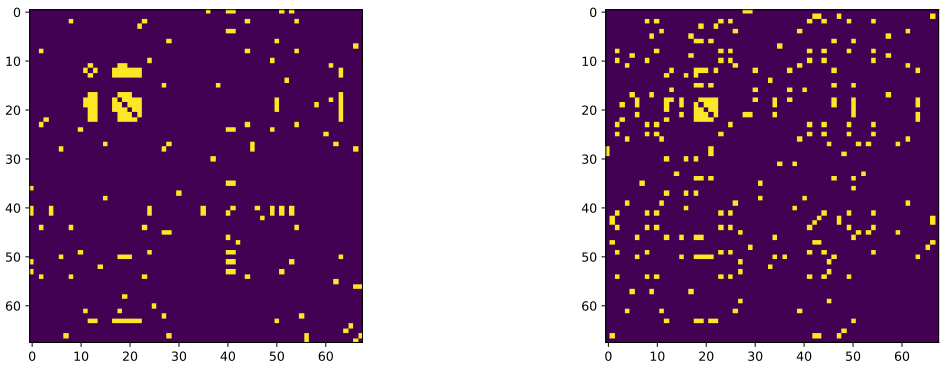
```
=== strategy='min_degree' (classical MIS, non-redundant basis) ===
Nodes: 68   Edges: 203   Isolated: 7
Basis size (MIS): 29  (42.6% of nodes)
Mean community size: 3.9   Max: 13
Non-basis nodes in >1 community: 27 / 39 (mean 2.15, max 6)

=== strategy='max_degree' (hub-first MIS) ===
Nodes: 68   Edges: 203   Isolated: 7
Basis size (MIS): 20  (29.4% of nodes)
Mean community size: 4.3   Max: 16
Non-basis nodes in >1 community: 16 / 48 (mean 1.38, max 3)

=== Top-5 non-basis metabolites by number of communities (min_degree) ===
  Glyceric acid           deg=15  nCommunities=6  communities: Pentanoic acid,
  ↪  Glycerol-3-phosphate, Serotonin, Threonic acid, Glycolic acid, Glx
  Erythronic acid         deg=14  nCommunities=5  communities: Pentanoic acid, Serotonin,
  ↪  Threonic acid, Glycolic acid, Glx
  Ser                     deg=15  nCommunities=4  communities: Gly, Val, leu, Phe
  Suberylglycine          deg= 5  nCommunities=3  communities: Pyroglutamate, Xylofuranose,
  ↪  Glx
  Pro                     deg=11  nCommunities=3  communities: Gly, Val, Tryp
```

With `strategy="min_degree"`, the algorithm selects 29 basis metabolites (43% of nodes), forming communities of mean size 3.9. Because non-basis metabolites can be adjacent to several basis nodes simultaneously, 27 of the 39 non-basis metabolites belong to more than one community. For example, Glyceric acid (degree 15) appears in 6 communities, revealing its role as a broad connector in the metabolite network.

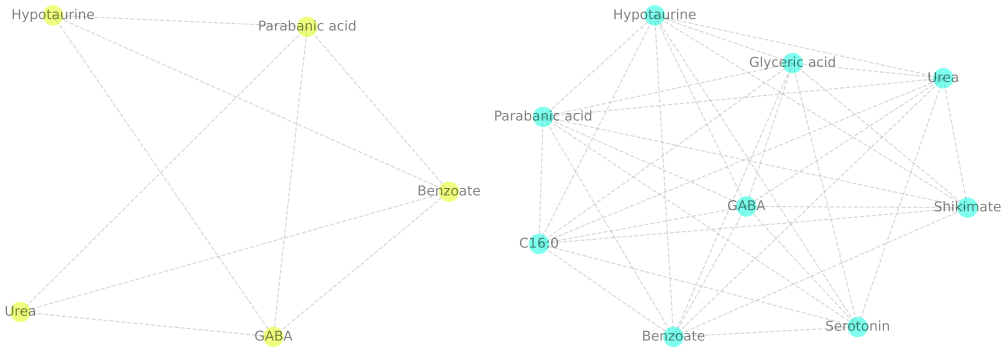
With `strategy="max_degree"`, only 20 basis nodes are selected. The hubs are placed in the basis first, producing larger communities (mean 4.3 nodes) but less overlap (maximum 3 communities per non-basis node). This strategy is analogous to `sphericalCaps` but guarantees that no two basis nodes are directly connected.



(a) Dry Seeds

(b) Six-hour Imbibed

Figure 4: Bitmap representation of the connectivity among metabolites.



(a) Dry Seeds (b) Six-hour Imbibed

Figure 5: The network component containing the GABA metabolite in each group.

4 Model Diagnostics

After fitting the model it is important to check the quality of the fit before interpreting the resulting network. Three diagnostic tools are provided.

`assessFit(res)` produces a 2×2 panel:

1. Fitted mixture histogram with the null (dashed), non-null, and combined density, plus the significance region.
2. Null Q-Q plot: high-posterior-null observations versus Beta$(\hat{\eta}, 0.5)$.
3. Non-null Q-Q plot: high-posterior-non-null observations versus Beta$(\hat{a}, \hat{b})$.
4. Model-based ROC curve, marking both the current threshold and the Youden-index optimum.

It also prints a verdict: *Good fit*, *Acceptable fit*, or *Deviations detected*. If the null Q-Q deviates substantially, refit with `ind=False` to let the effective sample size ν be estimated from the data.

`plotROC(res)` plots the ROC curve and returns `dict(auc=..., tau_youden=...)`. The Youden threshold maximises TPR$-$FPR and can be passed to `getAdjMat(res, ppthr=roc['tau_youden'])`.

`plotNonNullDensity(res)` shows the kernel density of non-null z_j values and reports the number of modes, which can reveal whether the non-null component is unimodal (a single correlation level) or multi-modal (several distinct association strengths in the data).

5 Community Detection

Once an adjacency matrix A is obtained from `getAdjMat()`, three complementary strategies are available for grouping nodes into communities.

5.1 Disjoint clusters: `graphComponents`

`graphComponents(A, minCtr=2)` assigns each node to exactly one cluster based on a centrality score (degree $\times$ clustering coefficient). Nodes with centrality below `minCtr` are left unclustered. This is the standard choice for data without strong hub structure.

5.2 Hub-and-spoke communities: `sphericalCaps`

`sphericalCaps(A)` builds overlapping communities by repeatedly selecting the highest-degree remaining node as a hub and assigning all its direct neighbours to its community. A node may appear in multiple communities if it is adjacent to several hubs. Best suited to networks with clear master-regulator hubs.

5.3 Maximal-independent-set basis: `greedyBasis`

`greedyBasis(A, strategy)` finds a *maximal independent set* (MIS) of the network: a set of nodes that are mutually non-adjacent (no two basis nodes are directly correlated) and maximal (every non-basis node is adjacent to at least one basis node). After finding the basis, each node is assigned to every community whose centre it is adjacent to in the original network, giving **overlapping membership**.

```
adjM  = bx.getAdjMat(res)
basis = bx.greedyBasis(adjM, strategy="min_degree", labels=M.columns)
bx.summarizeBasis(basis)
```

Two strategies are supported:

`"min_degree"` **(default)** The least-connected remaining node is selected at each step — classical greedy MIS. This produces a *non-redundant representative basis*: nodes that are as spread out in the graph as possible. Analogous to the MRB feature-selection basis used in statistical learning.

`"max_degree"` The most-connected remaining node is selected first — hub-first MIS. This produces fewer, larger communities and is conceptually similar to `sphericalCaps`, but with the strict guarantee that no two basis nodes are adjacent.

The returned DataFrame contains one row per node with columns: `isBasis`, `basisOrder`, `degree`, `cc`, `nCovered` (community size, for basis nodes), `nCommunities` (number of communities a node belongs to), and `communities` (list of basis-node indices). Nodes with `nCommunities` > 1 bridge multiple communities and often correspond to metabolites or genes with broad regulatory roles.

Table 1 summarises the key properties of the three approaches.

	graphComponents	**sphericalCaps**	**greedyBasis**
Membership	Disjoint	Overlapping	Overlapping
Basis nodes adjacent?	Yes	Yes	No (independent set)
Selection criterion	Max degree×cc	Max degree	Min or max degree
Best for	General clusters	Hub-spoke networks	Non-redundant basis

Table 1: Comparison of the three community-detection strategies in betaMixPy.

References

[1] Peter Frankl and Hiroshi Maehara. Some geometric applications of the beta distribution. *Annals of the Institute of Statistical Mathematics*, 42:463–474, 1990.

[2] Haim Bar and Martin T. Wells. On graphical models and convex geometry. *Computational Statistics & Data Analysis*, 187:107800, 2023. doi:10.1016/j.csda.2023.107800.

[3] Rashid Kazmi, Leo Willems, Ronny Joosen, Noorullah Khan, Wilco Ligterink, and Henk Hilhorst. Metabolomic analysis of tomato seed germination. *Metabolomics*, 13, 12 2017. doi10.1007/s11306-017-1284-x.